

b) Deadlock Detection

```
#include <stdio.h>
```

```
int main() {
    int n, m, i, j, k;
    printf("\nEnter number of processes: ");
    scanf("%d", &n);
    printf("\nEnter number of resources: ");
    scanf("%d", &m);
    int alloc[n][m], request[n][m];
    int avail[m];
    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);
    printf("\nEnter Request Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &request[i][j]);
    printf("\nEnter Available Resources:\n");
    for(i = 0; i < m; i++)
        scanf("%d", &avail[i]);
    int finish[n];
    for(i = 0; i < n; i++) {
        int empty = 1;
        for(j = 0; j < m; j++) {
            if(alloc[i][j] != 0) {
                empty = 0;
                break;
            }
        }
        finish[i] = empty;
    }
    int found;
    do {
        found = 0;
        for(i = 0; i < n; i++) {
            if(!finish[i]) {
                for(j = 0; j < m; j++) {
                    if(request[i][j] > avail[j])
                        break;
                }
                if(j == m) {
                    for(k = 0; k < m; k++)
                        avail[k] += alloc[i][k];
                }
            }
        }
    } while(found == 0);
}
```

```

        finish[i] = 1;
        found = 1;
    }
}
}

} while(found);
int deadlock = 0;
for(i = 0; i < n; i++) {
    if(!finish[i]) {
        deadlock = 1;
        printf("\nProcess P%d is in deadlock\n", i);
    }
}
if(!deadlock)
    printf("\nNo Deadlock Detected\n");
return 0;
}

```

OUTPUT:

```
Enter number of processes: 4
Enter number of resources: 3
Enter Allocation Matrix:
1 0 2
2 1 1
1 0 3
1 2 2
Enter Request Matrix:
0 0 1
1 0 2
0 0 0
3 0 2
Enter Available Resources:
0 0 0
No Deadlock Detected
Process returned 0 (0x0)   execution time : 37.488 s
Press any key to continue.
|
```